The 0 above means that Bob is the first (well, technically zeroth) element in the array. Continuing the example above, we can access the values of elements in $myArr3 individually:

```
$i = 1;

$output1 = $myArr3[0];
$output2 = $myArr3[$i];

print "Value of element 0 is {$output1}\n";
print "Value of element (\$i = 1) is {$output2}\n";

Output:
Value of element 0 is Bob
Value of element ($i = 1) is Alice
```

Here a literal integer value of 0 is used as the key to access the first element of $myArr3, and a variable (with an integer value of 1) is the key used to access the second element. PHP retrieves the values of the elements, which we then assign to other variables for use in subsequent statements (in this case, a print construct).

Array access sub-expressions will be coerced to integers if the array is numerically indexed. The example above demonstrates using variables as array indexes, but any expression that reduces to a value is legal syntax (provided it evaluates to an integer or a value that can be coerced). This includes values returned from function calls, as we can see in the following example:

```
$myArr = ['Alice', 'Bob'];

var_dump($myArr["0"]); // string is coerced to an int
var_dump($myArr["0" + "1"]); // operation can be coerced to an int
var_dump($myArr[2 - 1]); // expression evaluates to integer 1
var_dump($myArr[getArrayKey()]); // function returns integer 0

function getArrayKey(): int
{
    return 0;
}
```